

Game Loop Timing Architecture

Project: Umbra Engine **Date:** 2026-08-16 **Scope:** Fixed-timestep game loop, decoupled rendering, render interpolation

Executive Summary

Umbra's current game loop couples simulation and rendering to the same frame cadence and feeds raw `deltaTime` directly into `OnUpdate`. This produces framerate-dependent simulation — physics, movement, and collision results differ between machines and under frame spikes. The fix is a fixed-timestep accumulator loop that ticks simulation at exactly 1/60 s regardless of render cadence, with interpolated rendering to keep visuals smooth. This document covers the loop redesign, the render component changes needed to support interpolation, what stays the same, and the recommended migration order.

Problem: Current Loop

The existing loop in `Application::Run()` measures raw elapsed time each frame and passes it directly into `OnUpdate`, which propagates it through `SystemManager` and `SceneManager` to all game systems.

```
while (!window->ShouldClose()) {
    deltaTime = currentTime - lastTime;    // raw, noisy
    OnUpdate(deltaTime);                  // simulation sees variable step
    OnDebugUpdate();
    OnRender(renderer);                   // coupled to simulation tick
    renderer->EndDrawing();
}
```

Consequences:

- Simulation results are framerate-dependent. Movement with friction, acceleration, or collision detection produces different outcomes at 30 fps vs 60 fps vs 144 fps.
 - A single slow frame (OS event, shader compile, asset load) causes a large `deltaTime`, which causes objects to tunnel through geometry or overshoot targets.
 - No spiral-of-doom protection — a frame that takes longer than one tick triggers more updates, which makes the next frame slower, which triggers more updates.
 - Determinism is impossible. Input replays and frame-accurate debugging cannot be built on a variable-step loop.
-

Solution: Fixed Timestep with Decoupled Rendering

Simulation runs at a fixed rate of 1/60 s. Rendering runs as fast as the hardware allows, interpolating between the last two simulation states using the accumulator remainder as an alpha value.

Loop Structure

```
static constexpr float FIXED_TIMESTEP          = 1.0f / 60.0f;
static constexpr float MAX_ACCUMULATED_TIME    = 8.0f / 60.0f; // spiral cap (~7.5 fps)
static constexpr float TIME_SNAP_THRESHOLD     = 0.0002f;      // vsync jitter tolerance

float accumulator = 0.0f;

while (!window->ShouldClose()) {
    float rawDeltaTime = MeasureElapsedTime();
    float clampedDeltaTime = std::min(rawDeltaTime, MAX_ACCUMULATED_TIME);

    // Snap to exact fixed step when close – eliminates vsync jitter drift
    if (std::abs(clampedDeltaTime - FIXED_TIMESTEP) < TIME_SNAP_THRESHOLD) {
        clampedDeltaTime = FIXED_TIMESTEP;
    }

    accumulator += clampedDeltaTime;

    OnPollInput(); // every real frame, not every tick

    while (accumulator >= FIXED_TIMESTEP) {
        renderComponentManager->SnapshotAllPreviousStates();
        OnFixedUpdate(FIXED_TIMESTEP); // always exactly 1/60 s
        accumulator -= FIXED_TIMESTEP;
    }

    const float interpolationAlpha = accumulator / FIXED_TIMESTEP;

    OnDebugUpdate();
    OnRender(renderer, interpolationAlpha);
    OnDebugRender();
    renderer->EndDrawing();
}
```

Time Snapping

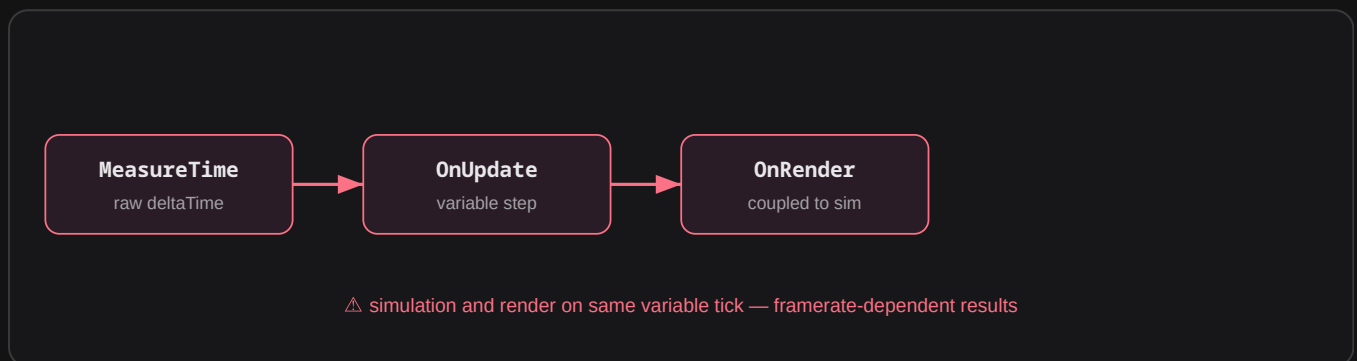
When vsync is active, frame times cluster very close to 1/60 s but never land exactly on it due to OS timer precision. Without snapping, the accumulator drifts, causing periodic double-updates (stutter). Snapping clamps measured delta to exactly `FIXED_TIMESTEP` when within `0.0002 s` — close enough to mean "this was a vsync frame." When vsync is off or the frame genuinely took longer, the raw value falls outside the threshold and passes through unchanged.

Spiral-of-Doom Cap

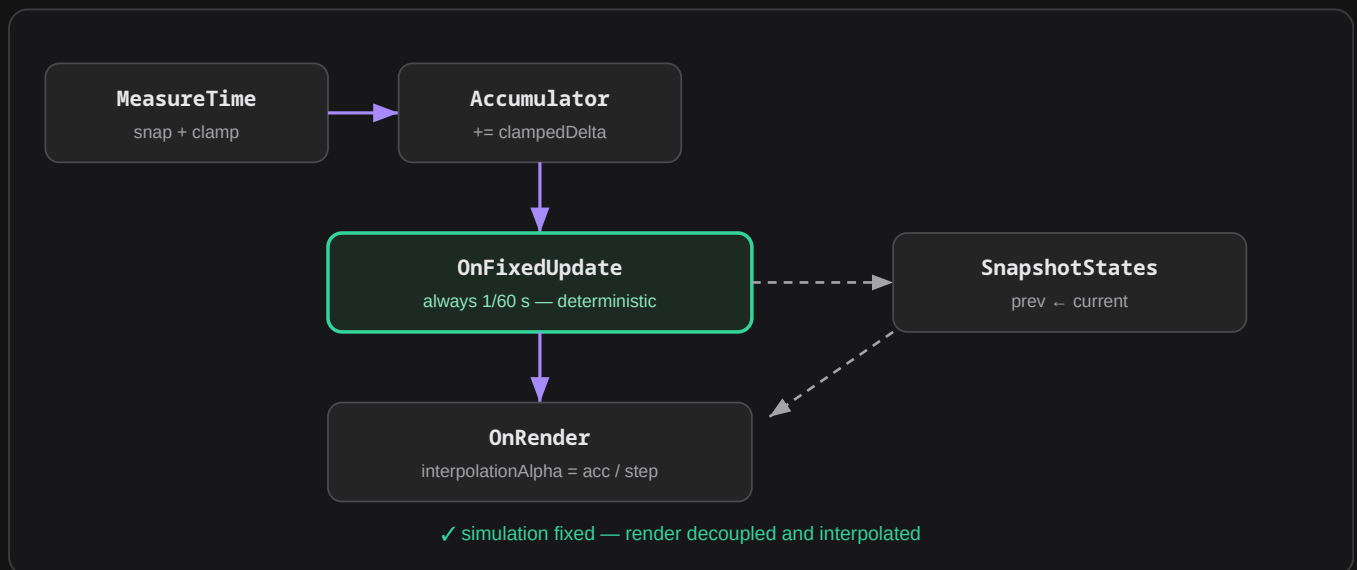
If a frame takes longer than `MAX_ACCUMULATED_TIME` (e.g. a blocking load), the accumulator is capped before entering the fixed-tick loop. The simulation falls behind real time rather than attempting to compensate with a burst of updates that would make subsequent frames even slower.

Data Flow Diagrams

Current Loop (Coupled)



Proposed Loop (Decoupled)



— purple solid: data flow | grey dashed: read/observe | green border: fixed-rate / deterministic

Render Component Changes

What Needs to Change

`RenderComponent2DManager::OnRender` currently calls `component->Render(renderer)` with no timing context. To interpolate, each component needs to know how far between its last two simulation states it is.

`IRenderComponentManager` — two new methods:

```
virtual void SnapshotAllPreviousStates() = 0;
virtual void OnRender(const IRenderer& renderer, float interpolationAlpha) const = 0;
```

`IRenderComponent2D` — snapshot contract:

```
class IRenderComponent2D : public IRenderComponent {
public:
    virtual void SnapshotPreviousState() = 0;
    virtual void Render(const IRenderer& renderer, float interpolationAlpha) const = 0;
};
```

Concrete component — dual transform:

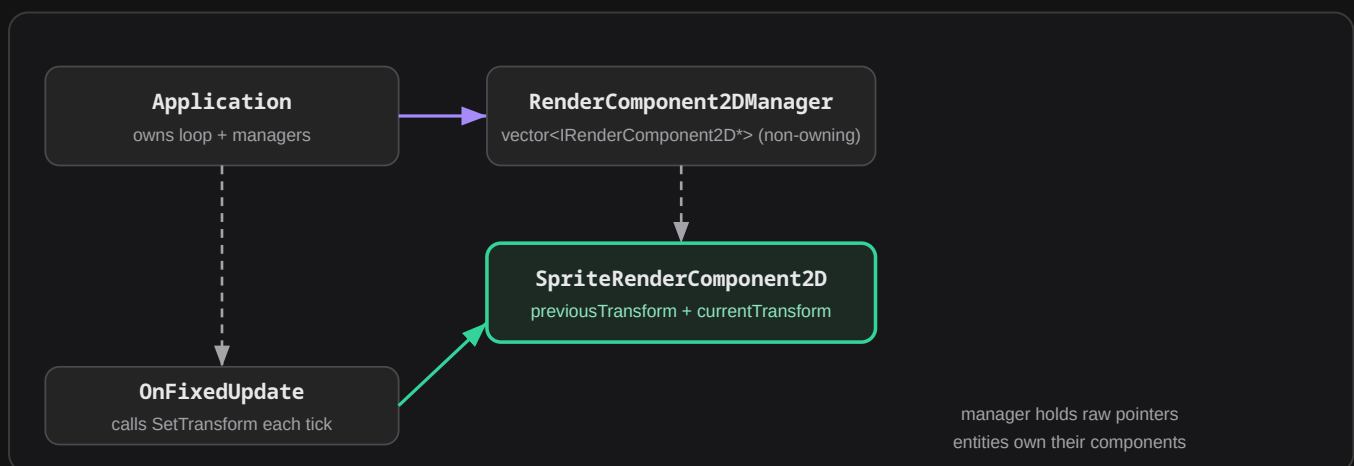
```

struct Transform2D {
    Vector2 position;
    float rotation;
    Vector2 scale;
};

class SpriteRenderComponent2D : public IRenderComponent2D {
    Transform2D previousTransform;
    Transform2D currentTransform;
public:
    void SnapshotPreviousState() override {
        previousTransform = currentTransform;
    }
    void SetTransform(const Transform2D& transform) {
        currentTransform = transform;
    }
    void Render(const IRenderer& renderer, float interpolationAlpha) const override {
        Vector2 interpolatedPosition = Vector2Lerp(
            previousTransform.position,
            currentTransform.position,
            interpolationAlpha
        );
        float interpolatedRotation = std::lerp(
            previousTransform.rotation,
            currentTransform.rotation,
            interpolationAlpha
        );
        renderer.DrawSprite(texture, interpolatedPosition, interpolatedRotation,
currentTransform.scale);
    }
};

```

Ownership and Data Flow



— purple solid: ownership | grey dashed: non-owning observer | green solid: writes simulation result | green border: dual-state component

What Does Not Change

Concern	Status	Notes
<code>IWindow</code> / <code>IRenderer</code> interfaces	Unchanged	Rendering call sites gain <code>alpha</code> param only
<code>InputSystem</code> poll path	Minor split	<code>PollInput</code> separated from action application
<code>EventBus</code>	Unchanged	Events fired inside <code>OnFixedUpdate</code> as before
<code>SceneManager</code> / <code>EntityManager</code>	Unchanged internally	Called via <code>OnFixedUpdate</code> instead of <code>OnUpdate</code>
<code>GameSystemManager</code>	Unchanged internally	Same — receives fixed step
<code>DebugHUD</code> / <code>OnDebugUpdate</code>	Unchanged	Runs once per real frame, reads live state
<code>SystemManager</code> signature	<code>OnUpdate(float)</code> → <code>OnFixedUpdate(float)</code>	Rename only — same float param
Raylib/SDL3 facade layer	Unchanged	Timing lives above the facade

Input: Poll vs Apply

Input polling must happen every real frame to avoid missing events between ticks. Input *application* (moving a character, consuming an action event) must happen inside `OnFixedUpdate` to stay deterministic.

Current `InputSystem::OnUpdate` likely does both. The split is:

```
OnPollInput()    – called once per real frame, before accumulator loop
                  reads hardware, updates ActionSet, fires raw input events

OnFixedUpdate()  – systems consume ActionSet state / events
                  movement and game logic respond here
```

No new interface is needed — `InputSystem` gains a `PollInput()` method and its `OnUpdate` (now `OnFixedUpdate`) only applies already-pollled state.

What to Not Interpolate

Not everything needs or should be interpolated. Apply the rule: **if it doesn't move smoothly between frames, interpolating it produces wrong results.**

State	Interpolate?	Reason
Entity position	Yes	Primary use case
Entity rotation	Yes, with care	Lerp breaks at $> 180^\circ$ delta — use <code>slerp</code> or clamp step size
Entity scale	No	Rarely changes frame-to-frame visually
Sprite frame index	No	Discrete — lerping an int makes no sense
Animation state machine	No	State transitions are instantaneous
Particle spawn events	No	1-tick delay unnoticeable
UI element position	Depends	Static UI: no. Animated UI that tracks world: yes
Audio triggers	No	Fire in <code>OnFixedUpdate</code> , not render path

Migration Order

1. **Restructure** `Application::Run()` — introduce accumulator, time snap, spiral cap. `OnUpdate` becomes `OnFixedUpdate`. All existing systems keep working; only the call site changes.
2. **Split** `InputSystem` — add `PollInput()`, move action application to `OnFixedUpdate`. Verify no input events are dropped.
3. **Add** `SnapshotAllPreviousStates()` to `IRenderComponentManager` — stub returns in the default implementation so nothing breaks.
4. **Add** `SnapshotPreviousState()` to `IRenderComponent2D` — stub initially copies `currentTransform` to `previousTransform` with no actual interpolation yet.
5. **Pass** `interpolationAlpha` through `OnRender` — plumb the parameter down to `RenderComponent2DManager::OnRender` and each component's `Render`.
6. **Implement interpolation in concrete render components** — activate `Vector2Lerp` / `std::lerp` in `SpriteRenderComponent2D::Render`. At this point interpolation is live.
7. **Validate** — test at locked 60 fps (should be identical to current behaviour), then at uncapped frame rate (should be smooth), then with an artificial slow frame injected (should not spiral).

Each step is independently shippable. Steps 1–2 improve correctness immediately even without interpolation. Steps 3–6 are additive.

Summary

Property	Before	After
Simulation timestep	Variable (raw <code>deltaTime</code>)	Fixed (always <code>1/60 s</code>)
Render rate	Coupled to simulation	Decoupled, runs at hardware rate
Frame smoothness	Dependent on vsync	Interpolated — smooth at any rate
Determinism	None	Full within fixed-tick boundary
Slow frame behaviour	Spiral of doom	Capped — simulation falls behind, does not freeze
Vsync jitter	Causes periodic double-updates	Eliminated by time snapping
Implementation cost	—	Loop: trivial. Interpolation: per-component, one-time